

# Texto Complementar - Aula 2

## De objeto isolado a cadastro em memoria

### 1. A evolucao da pratica

Na primeira pratica, o objetivo era compreender a estrutura basica de um programa Java: classe, objeto, metodo main e exibicao de dados. A Aula 2 avanca um passo: o programa passa a receber dados durante a execucao, guardar varios objetos e oferecer opcoes ao usuario.

### 2. Por que usar uma lista?

Um cadastro real raramente trabalha com apenas um objeto. Quando o usuario cadastra varios alunos, precisamos de uma estrutura para armazenar esses objetos enquanto o programa esta aberto. Em Java, uma opcao simples para esse momento e ArrayList.

```
private ArrayList<Aluno> alunos = new ArrayList<>();

public void adicionar(Aluno aluno) {
    alunos.add(aluno);
}
```

### 3. Separando responsabilidades

| Classe         | Responsabilidade                                                      |
|----------------|-----------------------------------------------------------------------|
| Aluno          | Representar os dados e comportamentos simples de um aluno.            |
| CadastroAlunos | Guardar a lista e executar operacoes como adicionar, listar e buscar. |
| Principal      | Interagir com o usuario e chamar as classes corretas.                 |

### 4. Busca por nome

Buscar um aluno significa percorrer a lista e comparar cada nome com o valor informado pelo usuario. Para comparar textos, usamos equalsIgnoreCase, e nao o operador ==.

```
for (Aluno aluno : alunos) {
    if (aluno.getNome().equalsIgnoreCase(nome)) {
        return aluno;
    }
}
return null;
```

### 5. Validacao simples

A validacao evita que o sistema aceite dados impossiveis ou vazios. Nesta etapa, vamos trabalhar com duas validacoes iniciais: texto obrigatorio e numero dentro de uma faixa.

- Texto obrigatorio: o usuario nao pode deixar o nome ou o curso em branco.
- Numero dentro de faixa: o semestre deve ficar entre 1 e 10.

- Erro de digitacao: se o usuario digitar texto no lugar de numero, o sistema deve avisar e pedir novamente.

## 6. JOptionPane e aplicacao desktop

JOptionPane e uma forma simples de criar interacao grafica em Java desktop. Ele nao substitui uma interface completa, mas permite que os estudantes percebam a diferenca entre imprimir no Console e conversar com o usuario por janelas.

### Mensagem central

A evolucao importante desta aula nao esta na janela em si: esta em transformar objetos soltos em um pequeno fluxo de cadastro com dados, lista, regras e resposta ao usuario.

## 7. Ligacao com as proximas aulas

- A lista em memoria sera substituida gradualmente por persistencia em banco.
- A classe CadastroAlunos antecipa a ideia de camada de servico.
- O menu com JOptionPane antecipa a construcao de telas mais organizadas.
- As validacoes simples preparam regras de negocio mais claras.